CNS Crypto CTF Cheat Sheet

Oct 29, 2026 · @Kyan

Quick triage

Match what the challenge gives you to the row below, then jump to that section. Hacking session 1 (29/10) covers Parts 1-3: classical ciphers, stream ciphers, block ciphers, hashes, MACs and passwords.

You see	Likely	First move
Letters only, spaces/punctuation kept	Caesar / substitution / Vigenère	`caesar_all`; if none reads, check IoC: ~0.066 = monoalphabetic (frequency analysis), lower = Vigenère (`vig_keylen`)
Ciphertext with repeated 16-byte blocks	AES-ECB	Block cut-and-paste, or byte-at-a-time if you control a prefix
Several ciphertexts, "same key", stream cipher / CTR / OFB / OTP	Keystream reuse	c1 ⊕ c2 = m1 ⊕ m2 → `many_time_pad` / `crib_drag`
A known plaintext + its ciphertext under a stream cipher	Keystream recovery	ks = c ⊕ m, decrypt the other ciphertext
Server decrypts a token, you want to change a field	Malleability	CTR/stream: flip c directly. CBC: flip IV / previous block (`cbc_flip`)
Server says "invalid padding" vs "invalid MAC"	CBC padding oracle	Byte-by-byte decryption via the last byte of C_(i−1)
Tag = SHA1/SHA256/MD5(secret ‖ data), you can append	Length extension	`sha256_extend` / hash_extender, brute-force key length 1-64
Tag = CBC-MAC of variable-length data	CBC-MAC forgery	MAC(X ‖ (X ⊕ T)) = T
Same GCM nonce on two messages	GCM nonce reuse	XOR ciphertexts like CTR; recover H to forge
32/40/64/128 hex chars, "password"	MD5 / SHA-1 / SHA-256 / SHA-512 hash	Crackstation, then `crack` with rockyou.txt; add the salt if given

You see	Likely	First move
Short or truncated hash (⩽ 32 bits)	Brute force / birthday	Preimage: loop all inputs. Collision: `birthday` (about 2^(n/2) tries)
Key is short (DES, "4-digit PIN", few unknown bytes)	Brute force	Try all keys, stop on the flag prefix or printable text
RC4 / WEP style key ‖ IV	RC4 key weaknesses	Reused key = reused keystream; first bytes leak the key
Hex/bytes you need to compute by hand (AES step, Salsa QR, SHA W_t)	Exercise-style	`gmul`, `mixcol`, `salsa_qr`, `chacha_qr` in kit.py

Security basics (Part 1)

The intro lecture is vocabulary, not attack material; know it for the flavour text of a challenge. Hacking session rules: individual, on lab PCs, notes and prepared code allowed, no LLM/GenAI. Session 1 (29/10) covers topics 1-4: history of crypto, stream ciphers, block ciphers, hashing, MACs and password storage.

Concept	Meaning	Typical mechanism
Confidentiality	Only authorised parties read the data	Encryption, access control
Integrity	Data changed only in an authorised way	MACs, digital signatures
Availability	Timely, reliable access, no DoS	Firewalls, redundancy
Authentication	Entity is who it claims to be	Authentication exchange
Nonrepudiation	Sender or receiver can't deny the message	Digital signatures, notarisation
Access control	Prevent unauthorised use of resources	ACLs

- **Passive attacks** (eavesdropping): release of message contents, traffic analysis (who talks to whom, how often, how long). Hard to detect, so prevent them with encryption.
- **Active attacks**: masquerade, replay, modification of messages, denial of service. Hard to prevent, so detect and recover.
- Breach lessons from the slides: hardcoded credentials in scripts (Uber 2022); unsalted SHA-1 password hashes (LinkedIn 2012), where identical passwords give identical hashes.

Math toolkit: finite fields

AES works in GF(2^8): bytes are polynomials with bit coefficients, addition is XOR, multiplication is reduced modulo m(x) = x^8 + x^4 + x^3 + x + 1 (0x11B).

- Hierarchy: groups ⊂ abelian groups ⊂ rings ⊂ commutative rings ⊂ integral domains ⊂ fields. A field supports +, −, ×, and ÷ by any non-zero element.
- Z_(2^n) is not a field: 2 has no inverse mod 2^n. GF(p) for prime p and GF(2^n) are fields.
- GF(2^n) element = a_(n−1)x^(n−1) + … + a_1x + a_0, each a_i ∈ {0,1}, so one n-bit number. For n = 3 the irreducible polys are x^3 + x^2 + 1 and x^3 + x + 1.
- **Addition = subtraction = XOR.** Example: 0xBA ⊕ 0x0B = 0xB1.
- **Multiplication**: multiply polynomials (coefficients mod 2), then take the remainder after dividing by m(x). Example: 0xBA × 0x0B → x^10 + x^4 + x^3 + x^2 + x → mod m(x) → x^6 + x^5 + x^4 + x = **0x72**.
- Shortcut xtime(a) = multiply by x (0x02): shift left 1; if bit 7 was set, XOR with 0x1B.
- **Inverse**: extended Euclid on polynomials. Example: $0x22^{-1}$ = **0x5A**.

```
r_{-1}=m(x),\; r_0=a(x),\; w_{-1}=0,\; w_0=1
```

```
q_i = r_{i-2} \,/\, r_{i-1},\quad r_i = r_{i-2} \bmod r_{i-1},\quad w_i =
w_{i-2} - q_i w_{i-1}
```

Stop when r_i = 1; then w_i = $a^{-1}(x)$. In GF(2^n) the minus is XOR.

Plain modular arithmetic for classical ciphers: a^{-1} mod n exists iff gcd(a, n) = 1; in Python `pow(a, -1, n)`.

Classical ciphers and how to break them

Every classical cipher in the course falls to brute force or frequency analysis; map letters a=0 … z=25.

Cipher	Encryption	Key space	Break it with
Caesar / shift	c = (p + k) mod 26	25 useful keys	Try all 25 shifts and pick the readable one
Monoalphabetic substitution	Each letter → fixed random letter	26! ≈ 4×10^26	Frequency analysis, then fix by hand with common words (the, and, ing)
Vigenère	c_i = (p_i + k_(i mod M)) mod 26	26^M	Find key length M, split into M Caesar ciphers, frequency-attack each
Rotor machines (Enigma)	Polyalphabetic, rotors step per letter	Large	Frequency attacks, cribs

English letter frequencies (%): E 12.7 · T 9.1 · A 8.2 · O 7.5 · I 7.0 · N 6.7 · S 6.3 · H 6.1 · R 6.0 · D 4.3 · L 4.0 · C 2.8 · U 2.8 · M 2.4 · W 2.4 · F 2.2 · G 2.0 · Y 2.0 · P 1.9 · B 1.5 · V 1.0 · K 0.8 · J 0.15 · X 0.15 · Q 0.10 · Z 0.07. Order to remember: **ETAOIN SHRDLU**.

Cracking Vigenère

1. Key length (Kasiski): repeated ciphertext sequences sit at distances that are multiples of M; take the gcd of those distances. Alternative: index of coincidence per column, about 0.066 for English and 0.038 for random.
2. Split: column j = letters at positions j, j+M, j+2M, … Each column is a Caesar cipher. Example: "arvsduklio" with M = 2 → "avdki" and "rsulo".
3. Per column, try all 26 shifts and pick the one whose letter counts best match English (chi-squared), or assume the most frequent letter is 'e'.
4. Decrypt: p_i = (c_i − k_(i mod M)) mod 26.

Slide exercise: key "pass", ciphertext "ihwscsowg" → subtract the key letter by letter.

Attacker models, weakest to strongest attacker: ciphertext only → known plaintext → chosen plaintext → chosen ciphertext → chosen text. A known file header (PNG `89 50 4E 47`, PDF `%PDF`, the flag prefix) counts as known plaintext.

Stream ciphers: OTP, RC4, Salsa20/ChaCha20

A stream cipher is c = m ⊕ G(k). Any repeated keystream, known plaintext or bit flip breaks it without touching the key.

One-time pad: key is truly random, as long as the message, used once. It is the only unconditionally secure cipher. Stream ciphers swap the random key for a PRG output, so they are at best computationally secure.

Attacks to try first

- **Key / nonce reuse (many-time pad)**: c1 ⊕ c2 = m1 ⊕ m2. Guess a word (" the ", the flag prefix) at every offset of c1 ⊕ c2 and see if the other plaintext turns readable (crib dragging). With many ciphertexts, solve column by column: per byte position try all 256 key bytes and score how much printable English comes out.
- **Known plaintext**: keystream = c ⊕ m. Reuse it on any other ciphertext under the same key/nonce.
- **Malleability (bit flipping)**: c′ = c ⊕ (old ⊕ new) decrypts to new without the key. Slide example: E(k, "1000€") ⊕ "1000€" ⊕ "9000€" = E(k, "9000€"). Defence is a MAC.
- **Predictable PRG**: if the first i keystream bits predict bit i+1, recover the keystream from a known header and roll it forward.

RC4 (Rivest 1987): 256-byte state S, byte-wise output.

```
def rc4(key: bytes, n: int) -> bytes:
    S = list(range(256)); j = 0
    for i in range(256):                        # KSA
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    i = j = 0; out = []
    for _ in range(n):                          # PRGA
        i = (i + 1) % 256; j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        out.append(S[(S[i] + S[j]) % 256])
    return bytes(out)
```

RC4 weaknesses: no nonce, so key reuse; first keystream bytes correlate with the key (WEP broke on key|IV); the 2nd output byte is 0 with probability 1/128 instead of 1/256; NOMORE attack (2014) on TLS and WPA-TKIP; banned in TLS by RFC 7465.

Salsa20 (Bernstein 2005): ARX design (add mod 2^32, rotate, XOR), constant-time. Input: 256-bit key, 64-bit nonce, 64-bit counter. Output: 64-byte blocks H(K, (N, P)) for P = 0, 1, 2, …

- State is 4×4 words, words little-endian: `C1 K1 K2 K3 / K4 C2 N1 N2 / P1 P2 C3 K5 / K6 K7 K8 C4`. Constant = "expand 32-byte k" ("expand 16-byte k" for 128-bit keys, which reuse K1..K4 as K5..K8).
- 20 rounds = 10 double rounds: columnround then rowround. Output = state after rounds + initial state (word-wise), since the rounds alone are invertible.
- Quarter round, in this order: y1 = x1 ⊕ ((x0 + x3) <<< 7); y2 = x2 ⊕ ((y1 + x0) <<< 9); y3 = x3 ⊕ ((y2 + y1) <<< 13); y0 = x0 ⊕ ((y3 + y2) <<< 18).

ChaCha20 (2008, RFC 7539): state `C1 C2 C3 C4 / K1..K4 / K5..K8 / P1 N1 N2 N3` (32-bit counter, 96-bit nonce). Even rounds work on diagonals instead of rows. Quarter round: a += b; d ^= a; d <<<= 16; c += d; b ^= c; b <<<= 12; a += b; d ^= a; d <<<= 8; c += d; b ^= c; b <<<= 7.

```
M = 0xffffffff
rotl = lambda x, n: ((x << n) | (x >> (32 - n))) & M
```

Block ciphers and modes of operation

AES itself is not the weak spot in a CTF; the mode, the IV/nonce handling or a tiny key space is.

DES / 3DES

- DES: 64-bit block, 56-bit key (8 parity bits ignored), 16-round Feistel, 48-bit round keys. Brute force is feasible: EFF's 1998 cracker took about 2 days.
- Feistel round: L_(i+1) = R_i, R_(i+1) = L_i ⊕ F(R_i, K_i). Decryption is the same network with the round keys reversed.
- 3DES = E(K3, D(K2, E(K1, P))). With K1 = K2 = K3 it is plain DES. 2-key: 112-bit key; 3-key: 168-bit key but about 112-bit strength (meet-in-the-middle).

AES (Rijndael, Daemen & Rijmen, KU Leuven): 128-bit block, state = 4×4 bytes filled **column by column**.

Key	Rounds	Expanded key
128 bit	10	44 words / 176 B
192 bit	12	52 words / 208 B
256 bit	14	60 words / 240 B

- Round 0: AddRoundKey. Rounds 1 to N−1: SubBytes → ShiftRows → MixColumns → AddRoundKey. Last round has no MixColumns.
- **SubBytes**: S-box lookup, high nibble = row, low nibble = column. S(0x95) = 0x2A, S(0x29) = 0xA5, IS(0x95) = 0xAD. No fixed points, not self-inverse.
- **ShiftRows**: row r rotates left by r bytes (row 0 stays).
- **MixColumns**: each column × matrix [[2,3,1,1],[1,2,3,1],[1,1,2,3],[3,1,1,2]] in GF(2^8). So s′0,0 = 2·s0 ⊕ 3·s1 ⊕ s2 ⊕ s3, with 3·a = xtime(a) ⊕ a. Inverse matrix uses 0E, 0B, 0D, 09.
- **AddRoundKey**: XOR the state with the 16-byte round key. It is the only step that uses the key.
- **Key expansion**: w[i] = w[i−4] ⊕ w[i−1]; for i mod 4 = 0 use w[i−4] ⊕ SubWord(RotWord(w[i−1])) ⊕ Rcon. RC = 01 02 04 08 10 20 40 80 1B 36. The expansion is invertible: any one full round key gives back the master key.

Modes

Mode	Encrypt	Decrypt	IV/nonce	What breaks it
ECB	C_i = E(P_i)	P_i = D(C_i)	none	Equal plaintext blocks → equal ciphertext blocks; cut-and-paste blocks; byte-at-a-time with a chosen-prefix oracle
CBC	C_i = E(P_i ⊕ C_(i−1)), C_0 = IV	P_i = D(C_i) ⊕ C_(i−1)	unpredictable IV	Flip bit in C_(i−1) or IV → same bit flips in P_i (block i−1 garbles); predictable IV; padding oracle
CFB	C_i = P_i ⊕ MSB_s(E(I_i)), register shifts in C_i	P_i = C_i ⊕ MSB_s(E(I_i))	IV	Bit errors propagate into following segments; uses only E
OFB	O_i = E(O_(i−1)), C_i = P_i ⊕ O_i	same XOR	unique nonce	Keystream independent of plaintext: nonce reuse = many-time pad; bit flipping
CTR	C_i = P_i ⊕ E(nonce + i)	same XOR	unique counters	Counter reuse = many-time pad; bit flipping; known P_i gives E(ctr_i)
XTS	C = E(K1, P ⊕ T) ⊕ T, T = E(K2, i) ⊗ α^j	mirror with D	sector i, block j	For disk storage; same block at same location always encrypts the same

CFB, OFB and CTR only use the encryption function, even for decryption.

Recipes

- **Spot ECB**: split ciphertext in 16-byte blocks; any duplicate means ECB (or feed it 48 identical bytes).
- **ECB byte-at-a-time** (oracle returns E(input ‖ secret)): send 15 × 'A', note block 0; then try 'A'×15 + every byte b until block 0 matches; that b is the first secret byte. Shrink the pad by one and repeat.
- **CBC bit flip**: to change P_i[k] from a to b, set C_(i−1)[k] ^= a ^ b. For the first block, edit the IV.

- **CBC padding oracle** (beyond the slides, common in CTFs): with PKCS#7 and a server that says "bad padding", tweak the last byte of C_(i−1) until the padding is valid; that leaks D(C_i) one byte at a time.
- **Small key space**: if only a few key bytes are unknown, brute force them and check for the flag prefix or printable output.

Hashes, MACs and password storage (Part 3)

The CTF-relevant facts: H(key ‖ msg) with SHA-1/SHA-2/MD5 is length-extendable, unsalted hashes fall to lookup tables, and collisions cost only 2^(m/2).

Properties

Property	Meaning	Brute-force cost (m-bit hash)	Needed for
Preimage resistant (one-way)	Given h, hard to find y with H(y) = h	2^(m−1) on average	MACs, signatures, password files
Second preimage resistant	Given x, hard to find y ≠ x with H(y) = H(x)	2^(m−1)	MACs, signatures, file integrity
Collision resistant	Hard to find any x ≠ y with H(x) = H(y)	2^(m/2) (birthday)	Signatures under chosen-message attack

- Birthday paradox: among √N random values, a repeat is more likely than not. 23 people → >50% shared birthday.
- Yuval's attack: make 2^(m/2) harmless variants x′ and fraudulent variants y′ (e.g. 1 vs 2 spaces between words: 33 words give 2^32 variants), find H(x′) = H(y′), get x′ signed, ship y′.
- Truncated or tiny hashes in a challenge (16-32 bits): brute force a preimage directly, or birthday a collision with a dict.

SHA family

Hash	Output	Block	Word	Rounds	Length extension?
MD5	128	512	32	64	Yes
SHA-1	160	512	32	80	Yes
SHA-224 / 256	224 / 256	512	32	64	Yes (224 truncated: much harder)
SHA-384 / 512	384 / 512	1024	64	80	Yes (384 truncated: much harder)
SHA-3 (Keccak)	224-512	r = 1152/1088/832/576	64	24	No (sponge, hidden capacity c)

- Merkle-Damgård: pad with 0x80, zeros, then the bit length (64-bit for SHA-256, 128-bit for SHA-512, big-endian) up to a block multiple; H_i = H_(i−1) + F(H_(i−1), M_i). The final state is the digest.

- SHA-512 round: T1 = h + Ch(e,f,g) + Σ1(e) + W_t + K_t; T2 = Σ0(a) + Maj(a,b,c); then shift a…h and set e = d + T1, a = T1 + T2. Ch = (e∧f)⊕(¬e∧g), Maj = (a∧b)⊕(a∧c)⊕(b∧c), Σ0 = ROTR28⊕ROTR34⊕ROTR39, Σ1 = ROTR14⊕ROTR18⊕ROTR41.
- Message schedule: W_t = σ1(W_(t−2)) + W_(t−7) + σ0(W_(t−15)) + W_(t−16), with σ0 = ROTR1⊕ROTR8⊕SHR7, σ1 = ROTR19⊕ROTR61⊕SHR6 for SHA-512. In the slides' 8-bit toy version, W16 = 0x78.
- SHA-3 sponge: state b = r + c = 1600 bits (5×5 lanes of 64 bits); absorb r-bit blocks by XOR + Keccak-f, then squeeze. Padding pad10*1.

Length extension attack (on MAC = H(K ‖ M) with MD5/SHA-1/SHA-2)

1. Known: M, tag = H(K ‖ M), the hash algorithm, and the key length (or brute force it, try 1-64).
2. Build glue = MD padding for a message of len(K) + len(M) bytes.
3. Load the tag as the internal state, set the processed length to len(K ‖ M ‖ glue), and hash your extension.
4. Submit M ‖ glue ‖ extension with the new tag. Slide example: `waffle=eggo` + padding + `&waffle=liege`, key length 14 bytes, length field 0x228 = (14 + 55) × 8.

Tools: `hash_extender`, `hashpump`, or the Python `hlextend` package. HMAC and SHA-3 are not vulnerable.

MACs

- **CBC-MAC / DAA** (raw, no subkeys): if T = MAC(K, X) for one block X, then MAC(K, X ‖ (X ⊕ T)) = T too. Forge any variable-length message this way.
- **CMAC**: CBC-MAC with a final tweak. L = E(K, 0^b), K1 = L·x, K2 = L·x² in GF(2^b). Full last block uses K1, padded (10…0) last block uses K2.
- **HMAC**: HMAC(K, M) = H((K⁺ ⊕ opad) ‖ H((K⁺ ⊕ ipad) ‖ M)), ipad = 0x36 repeated, opad = 0x5C repeated, K⁺ = key zero-padded to the block size (hashed first if longer). Breaking it needs a compression-function break or 2^(n/2) observed online messages.
- Brute-forcing a k-bit MAC key from one (M, tag) pair with n < k leaves about 2^(k−n) candidate keys, so you need several pairs.
- Compare tags in constant time; a byte-by-byte early-exit compare is a timing oracle.

Authenticated encryption

Approach	Construction	Used by	Note
Hash then encrypt	E(K, M ‖ H(M))	WEP	Broken; with a stream cipher you can flip bits and fix a linear checksum (CRC)
MAC then encrypt	E(K2, M ‖ MAC(K1, M))	SSL/TLS	Decrypt before verify: padding-oracle risk
Encrypt then MAC	C = E(K2, M), T = MAC(K1, C)	IPsec, GCM	Verify before decrypt: the preferred order

Approach	Construction	Used by	Note
Encrypt and MAC	C = E(K2, M), T = MAC(K1, M)	SSH, CCM	Tag can leak equal plaintexts

- **CCM**: AES-CTR + CBC-MAC, one key, nonce N, associated data A.
- **GCM**: H = E(K, 0^128); J0 = IV ‖ 0^31 ‖ 1 for a 96-bit IV; C = GCTR(inc32(J0), P); T = GCTR(J0, GHASH(A ‖ C ‖ lengths)). GHASH = polynomial in H over GF(2^128). **Reusing a nonce** leaks the CTR keystream and lets you recover H, then forge tags.

Password storage and cracking

- Store salt + SlowHash(salt ‖ password) (bcrypt, scrypt, PBKDF2, Argon2). Salt hides duplicate passwords, defeats precomputed tables, and hides reuse across sites.
- **Dictionary attack**: hash every wordlist entry (with every salt) and compare. **Rainbow table**: precomputed hash → password table; a b-bit salt multiplies its size by 2^b.
- Unsalted fast hashes: try crackstation.net, hashcat or john with rockyou.txt. LinkedIn 2012: SHA-1("123456") = 7c4a8d09ca3762af61e59520943dc26494f8941b.
- Hash identification by hex length: 32 = MD5/NTLM, 40 = SHA-1, 64 = SHA-256/SHA3-256, 128 = SHA-512. `$2b$` = bcrypt, `$6$` = sha512crypt, `$1$` = md5crypt, `$argon2id$` = Argon2.

Python toolkit (kit.py)

All helpers are in kit.py, sent with this doc. Each one was tested against a known answer: RFC/spec test vectors for RC4, Salsa20 and ChaCha20, the slides' GF(2^8) examples, and a working length extension. Only the AES-based recipes need `pycryptodome`; everything else is standard library. Run `from kit import *` in a Python shell.

Function	Use it for
`xor(a, b)`	XOR / repeating-key XOR
`caesar_all(ct)`, `shift(t, k)`	Print all 26 shifts; shift by k
`vig_keylen(ct)`, `vig_crack(ct, m)`, `vigenere(t, key)`	IoC per key length (look for ~0.066), crack with length m, decrypt
`single_byte_xor(ct)`	Best single-byte XOR key by English score
`many_time_pad(cts)`, `crib_drag(c1, c2, crib)`	Reused stream keystream
`rc4(key, n)`, `salsa_qr(...)`, `chacha_qr(...)`	Recompute keystreams and quarter rounds by hand
`gmul(a, b)`, `ginv(a)`, `mixcol(col)`	GF(2^8) multiply, inverse, AES MixColumns

Function	Use it for
`is_ecb(ct)`, `ecb_byte_at_a_time(oracle)`	Detect ECB; recover the secret behind an ECB oracle
`cbc_flip(prev, known, wanted, offset)`	CBC bit flipping via the IV or previous block
`pkcs7(b)`, `unpad(b)`, `blocks(b)`	Padding and splitting
`md_pad(n)`, `sha256_extend(tag, keylen, msg, ext)`	Length extension on SHA-256(key ‖ msg)
`crack(hash, wordlist, algo, salt)`, `birthday(f, bits)`	Wordlist cracking; truncated-hash collisions

Slide exercise answers (computed): Vigenère "ihwscsowg" with key "pass" → **theanswer**. Salsa20 QR(E4971786, 0096E9C1, 932F7534, 27F0D9BB) → y0..y3 = **A1477CD4, 436E4947, 99EEEF64, 80E5A210**. GF(2^8): 0xBA × 0x0B = 0x72, $0x22^{-1}$ = 0x5A.

```
# AES with pycryptodome
from Crypto.Cipher import AES
AES.new(key, AES.MODE_ECB).decrypt(ct)
AES.new(key, AES.MODE_CBC, iv).decrypt(ct)
AES.new(key, AES.MODE_CTR, nonce=nonce).decrypt(ct)      # pycryptodome:
nonce + counter split
AES.new(key, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)

# conversions
bytes.fromhex('41'); b'A'.hex(); int.from_bytes(b, 'big'); n.to_bytes(16,
'big')
import base64; base64.b64decode(s); base64.b32decode(s)
import hmac, hashlib; hmac.new(k, m, hashlib.sha256).hexdigest()
```

Other tools if installed: CyberChef ("Magic" recipe for unknown encodings), hashcat / john with rockyou.txt, hash_extender / hashpump, quipqiup.com for substitution ciphers. Note: the lab rules forbid LLM/GenAI tools during the hacking session; this doc and kit.py count as notes and prepared code.

Appendix

Searchable reference: Ctrl+F any term, constant, hex value or number.

A. Glossary

Term	Meaning
Access control	Prevention of unauthorized use of resources (X.800 security service)

Term	Meaning
Active attack	Modifies data or creates false data: masquerade, replay, modification of messages, denial of service. Hard to prevent, so detect and recover
AddRoundKey	AES step: XOR state with the 128-bit round key; the only step that uses the key
AE / Authenticated encryption	Encryption that protects confidentiality and integrity together (H→E, A→E, E→A, E+A, CCM, GCM)
AES	Advanced Encryption Standard: Rijndael (Daemen & Rijmen, KU Leuven), chosen 2000, published 2001; 128-bit block; 128/192/256-bit key; 10/12/14 rounds
ARX	Add-Rotate-XOR: 32-bit modular addition, fixed rotation, XOR. Fast, constant time (no timing attacks). Used by Salsa20/ChaCha20
Associated data (A)	Data that is authenticated but not encrypted, e.g. protocol headers (CCM, GCM)
Authentication	Assurance that the communicating entity is who it claims to be
Authenticator	Value used to authenticate a message: a hash, a ciphertext, or a MAC
Availability	Timely, reliable access with no denial of service to authorized users (CIA)
Birthday paradox	Among √N uniform random values a repeat is more likely than not; 23 people → >50% shared birthday; collision on m-bit hash ≈ 2^(m/2)
Block cipher	Encrypts fixed-size blocks of b bits as a whole (DES 64, AES 128)
Brute-force attack	Try every key; on average half the key space
CBC	Cipher Block Chaining: $C_i = E(K, P_i \oplus C_{(i-1)})$, $C_0 = IV$
CBC-MAC / DAA	Data Authentication Algorithm: last CBC block as tag; insecure for variable-length messages
CCM	Counter with CBC-MAC: AES + CTR + CMAC, single key; encrypt-and-MAC; NIST SP 800-38C; Wi-Fi WPA2
CFB	Cipher Feedback: stream mode on s-bit segments; $C_i = P_i \oplus MSB_s(E(K, I_i))$; bit errors propagate

Term	Meaning
ChaCha20	Bernstein 2008 variant of Salsa20, RFC 7539; 32-bit counter, 96-bit nonce, diagonal rounds, better diffusion
Chosen ciphertext	Attacker gets plaintexts for ciphertexts of their choice
Chosen plaintext	Attacker gets ciphertexts for plaintexts of their choice (e.g. differential cryptanalysis)
Chosen text	Chosen plaintext and chosen ciphertext together
CIA triad	Confidentiality, Integrity, Availability (NIST definition of computer security)
Ciphertext only	Attacker knows only algorithm + ciphertext; hardest for the attacker
Ciphertext stealing	XTS trick for a partial last block: borrow bits from the previous ciphertext block instead of padding
CMAC	Cipher-based MAC (NIST SP 800-38B): CBC-MAC with subkeys K1 = L·x, K2 = L·x², L = E(K, 0^b)
Collision resistant	Hard to find any x ≠ y with H(x) = H(y); "strong" hash
Compression function	The per-block function F in Merkle-Damgård hashing
Computational / semantic security	Breaking costs more than the information is worth, or takes longer than its lifetime;
Confidentiality	Preserving authorized restrictions on access and disclosure; data confidentiality + privacy
Confusion / diffusion	Hide key–ciphertext relation / spread each input bit over many output bits
Cryptanalysis	Attack using the algorithm's structure and plaintext properties
Cryptology	Cryptography + cryptanalysis
CTR	Counter mode: C_i = P_i ⊕ E(K, ctr_i); parallel, random access, preprocessing, only E needed
Denial of service	Preventing normal use of communication facilities (active attack); e.g. grey hole router dropping packets
DES	Data Encryption Standard, NIST 1977; 64-bit block, 56-bit key, 16-round Feistel, 48-bit round keys

Term	Meaning
Dictionary attack	Hash every candidate password (with every salt) and compare to the password file
Digital signature	Authentication that also counters source repudiation
Double round	Salsa20: columnround + rowround (counts as two rounds)
E→A (encrypt then MAC)	C = E(K2, M), T = MAC(K1, C); IPsec; verify before decrypt
E+A (encrypt and MAC)	C = E(K2, M), T = MAC(K1, M); SSH
A→E (MAC then encrypt)	E(K2, M ‖ MAC(K1, M)); SSL/TLS
H→E (hash then encrypt)	E(K, M ‖ H(M)); WEP; fundamentally flawed
ECB	Electronic Codebook: each block encrypted independently; equal blocks → equal ciphertext
Enigma	German WWII rotor machine (polyalphabetic), broken by the Allies
Feistel cipher	L_(i+1) = R_i, R_(i+1) = L_i ⊕ F(R_i, K_i); DES
Field	Set with +, −, ×, ÷ (non-zero); GF(p), GF(2^n)
GCM	Galois/Counter Mode, NIST SP 800-38D: GCTR + GHASH; encrypt-then-MAC; GMAC = auth only
GF(2^n)	Finite field of n-bit polynomials; add = XOR; multiply mod irreducible m(x)
GHASH	Keyed hash in GF(2^128): Y_i = (Y_(i−1) ⊕ X_i)·H, H = E(K, 0^128)
Grey hole attack	Router drops a subset of messages: denial of service
HMAC	H((K⁺⊕opad) ‖ H((K⁺⊕ipad) ‖ M)); RFC 2104, FIPS 198; IPsec, TLS
Index of coincidence	Probability two random letters are equal: English ≈ 0.066, random ≈ 0.038
Integrity	Guarding against improper modification or destruction; data integrity (MACs) + system integrity
Irreducible polynomial	Cannot be factored into two lower-degree polynomials over the field
IV	Initialization vector: CBC/CFB need it unpredictable; OFB/CTR need a unique nonce

Term	Meaning
Kasiski examination	Distances between repeated ciphertext sequences are multiples of the Vigenère key length
Keccak	SHA-3 winner (2012), FIPS 202 (2015); sponge with Keccak-f[1600], 24 rounds
Kerckhoffs's principle	Assume the attacker knows the algorithm; only the key is secret
Key expansion	AES: 128-bit key → 44 words; RotWord, SubWord, Rcon
Known plaintext	Attacker has plaintext–ciphertext pairs (headers, standard banners, probable words)
Length extension attack	From H(K‖M) and len(K‖M) compute H(K‖M‖pad‖M′) without K; MD5, SHA-1, SHA-2; not SHA-3/HMAC
MAC	Message Authentication Code: fixed-size C(K, M); integrity + origin, no nonrepudiation
Malleability	Ciphertext can be changed into one that decrypts to a related plaintext (stream ciphers, CTR, OFB, partly CBC)
Masquerade	One entity pretends to be another (active attack)
Merkle-Damgård	Iterated hash: pad + length, H_i = F(H_(i−1), M_i); MD4, MD5, SHA-0/1/2
Message authentication	Verify received messages were not altered: no modification, insertion, deletion or replay
MixColumns	AES step: each column × fixed matrix in GF(2^8); omitted in last round
Monoalphabetic cipher	One fixed substitution alphabet; 26! keys; broken by frequency analysis
Nonce	Number used once
NOMORE	2014 RC4 attack on TLS and WPA-TKIP using keystream biases (~75 h)
Nonrepudiation	Neither sender nor receiver can deny the message
Notarization	Trusted third party assures properties of an exchange (specific mechanism)
OFB	Output Feedback: O_i = E(K, O_(i−1)), C_i = P_i ⊕ O_i; no error propagation; nonce must be unique

Term	Meaning
One-time pad	Vernam 1917: random key as long as the message, used once; unconditionally secure
opad / ipad	HMAC constants 0x5C / 0x36 repeated b/8 times
Passive attack	Eavesdropping: release of message contents, traffic analysis. Hard to detect, so prevent (encryption)
Pervasive mechanisms	Trusted functionality, security labels, event detection, security audit trail, security recovery
Polyalphabetic cipher	Several substitution alphabets (Vigenère, rotor machines)
Preimage resistant	One-way: given h, hard to find y with H(y) = h
PRG	Pseudorandom generator G: {0,1}^s → {0,1}^n, n ≫ s; must be unpredictable
Product cipher	Multiple stages of substitution and transposition
Rainbow table	Precomputed hash → password table; trades time for space; a b-bit salt multiplies size by 2^b
RC4	Rivest 1987 byte stream cipher; 256-byte state; WEP; prohibited in TLS (RFC 7465, 2015)
Rcon	AES round constant word (RC[j], 0, 0, 0)
Replay	Capture data and retransmit for an unauthorized effect
Rijndael	Original name of AES
Rotor machine	Mechanical polyalphabetic cipher (1870–1970)
Salsa20	Bernstein 2005 ARX stream cipher; 256-bit key, 64-bit nonce, 64-bit counter, 20 rounds
Salt	Random value stored with a password hash; hides duplicates, slows dictionary attacks, hides reuse
Second preimage resistant	Given x, hard to find y ≠ x with H(y) = H(x); "weak" hash = first five properties
Security attack / mechanism / service	X.800: action compromising security / process to detect, prevent or recover / service using mechanisms
SHA	Secure Hash Algorithm: SHA-0 (1993), SHA-1 (1995), SHA-2 (2002), SHA-3 (2012/2015)
ShiftRows	AES step: row r rotated left by r bytes

Term	Meaning
Specific mechanisms	Encipherment, digital signature, access control, data integrity, authentication exchange, traffic padding, routing control, notarization
Sponge construction	Absorb r-bit blocks into b = r + c state, then squeeze output; SHA-3
Stream cipher	Encrypts one element (bit/byte) at a time: c = m ⊕ G(k)
SubBytes	AES step: S-box lookup (high nibble = row, low nibble = column)
Substitution / transposition	Replace elements / rearrange elements
Traffic analysis	Observing who communicates, how often, how long (passive)
Traffic padding	Insert bits into gaps to frustrate traffic analysis
Triple DES (3DES)	E(K3, D(K2, E(K1, P))); 2-key 112-bit, 3-key 168-bit (~112-bit strength)
Tweakable block cipher	C = E(K, T, P); tweak T is public and adds variability
Unconditional / perfect security	Ciphertext reveals nothing about the plaintext however much is available; only OTP
Vigenère	Polyalphabetic: c_i = (p_i + k_(i mod M)) mod N
WEP	Wi-Fi security using RC4 with weak key generation; hash-then-encrypt
X.800	ITU-T Security Architecture for OSI: attacks, mechanisms, services
xtime	Multiply by x (0x02) in GF(2^8): shift left, XOR 0x1B on carry
XTS-AES	XEX Tweaked-codebook with ciphertext Stealing; IEEE 1619-2007, NIST SP 800-38E 2010; disk storage

B. Sizes, key spaces and attack costs

Ciphers

Cipher	Type	Block	Key	Rounds	Key space / strength	Status (slides)
Caesar	Substitution	1 letter	shift k	—	25 (26 incl. k = 0)	Brute force instantly

Cipher	Type	Block	Key	Rounds	Key space / strength	Status (slides)
Monoalphabetic	Substitution	1 letter	permutation	—	26! ≈ 4.03×10^26 ≈ 2^88.4	Frequency analysis
Vigenère	Polyalphabetic	1 letter	M letters	—	26^M	Kasiski + frequency
One-time pad	Vernam	bit	= message length	—	2^len	Unconditionally secure, impractical
RC4	Stream	byte	8–2048 bit (1–256 B)	—	period > 10^100	Insecure (biases, no nonce)
Salsa20	Stream (ARX)	64-byte keystream block	256 (or 128)	20	2^256; 8-round variant broken in 2^250	Secure
ChaCha20	Stream (ARX)	64-byte keystream block	256	20	2^256	Secure
DES	Feistel	64	56 (+8 parity)	16	2^56 ≈ 7.2×10^16	Brute-forceable
2-key 3DES	EDE	64	112	48	2^112	Retire (NIST SP 800-57)
3-key 3DES	EDE	64	168	48	~2^112 effective	Computationally secure
AES-128	SPN	128	128	10	2^128	Secure
AES-192	SPN	128	192	12	2^192	Secure
AES-256	SPN	128	256	14	2^256	Secure

- AES expanded key: 44 words / 176 B (128), 52 words / 208 B (192), 60 words / 240 B (256). Round key always 128 bit.
- DES round key 48 bit; F does expansion, S-box substitution, permutation, XOR with round key.
- Ideal 4-bit block cipher: 16! mappings → minimum key log2(16!) ≈ 44.25 → **45 bits**; storing the table naively takes 16 × 4 = 64 bits.
- Brute force averages half the key space: 2^(k−1) tries. 2^56 keys at 10^9 keys/s ≈ 2.3 years worst case, ≈ 1.1 years average.
- DES brute-force history: 1977 Diffie & Hellman propose $20M machine (1 day); 1993 Wiener: $1M, 7 hours; 1997 DESCHALL wins $10,000 prize with idle Internet PCs; 1998 EFF cracker $250,000, about 2 days.

Salsa20 / ChaCha20 inputs

	Salsa20	ChaCha20 (RFC 7539)
Constant	128 bit "expand 32-byte k"	same
Key	256 bit (128-bit key duplicated)	256 bit
Nonce	64 bit	96 bit
Counter	64 bit	32 bit
State layout	C1 K1 K2 K3 / K4 C2 N1 N2 / P1 P2 C3 K5 / K6 K7 K8 C4	C1 C2 C3 C4 / K1 K2 K3 K4 / K5 K6 K7 K8 / P1 N1 N2 N3
Rotations	7, 9, 13, 18	16, 12, 8, 7
Even rounds on	rows	diagonals
Diffusion per QR	~8 output bits per input bit	~12.5

Hashes

Hash	Digest	Block	Word	Rounds	Max message	Length field	Collision (2^(n/2))	Preimage
MD5	128	512	32	64	—	64 bit LE	2^64 (broken)	2^128
SHA-1	160	512	32	80	< 2^64	64 bit BE	2^80 nominal, 2^69 attack (2005), broken	2^160
SHA-224	224	512	32	64	< 2^64	64 bit	2^112	2^224
SHA-256	256	512	32	64	< 2^64	64 bit	2^128	2^256
SHA-384	384	1024	64	80	< 2^128	128 bit	2^192	2^384
SHA-512	512	1024	64	80	< 2^128	128 bit	2^256	2^512
SHA-512/224	224	1024	64	80	< 2^128	128 bit	2^112	2^224
SHA-512/256	256	1024	64	80	< 2^128	128 bit	2^128	2^256
SHA3-224	224	r = 1152, c = 448	64	24	none	—	2^112	2^224
SHA3-256	256	r = 1088, c = 512	64	24	none	—	2^128	2^256
SHA3-384	384	r = 832, c = 768	64	24	none	—	2^192	2^384
SHA3-512	512	r = 576, c = 1024	64	24	none	—	2^256	2^512

- SHA-224 = SHA-256 with a different IV, truncated to the leftmost 224 bits. SHA-384, SHA-512/224 and SHA-512/256 = truncated SHA-512 (different IVs).

- SHA-3: r + c = 1600 bits; state 5×5 lanes × 64 bits; default Keccak c = 1024, r = 576. Larger c = more security, less speed.
- SHA history: 1993 SHA-0 (FIPS 180), 1995 SHA-1 (FIPS 180-1), 2002 SHA-2 (FIPS 180-2), 2005 NIST plans to phase out SHA-1, 2008 SHA-224 (FIPS 180-3), 2012 SHA-3 winner, 2015 FIPS 202 + FIPS 180-4 (SHA-512/t).

Generic attack costs

Goal	Cost
Brute-force a k-bit key	2^k worst, 2^(k−1) average
Preimage / second preimage, m-bit hash	2^(m−1) average
Collision, m-bit hash	≈ 2^(m/2) (64-bit hash → 2^32 variants)
MAC key brute force, k-bit key, n-bit tag, one pair	2^k work, leaves ≈ 2^(k−n) candidate keys
MAC key, number of pairs needed	≈ ⌈k / n⌉ rounds (k = 24, n = 16 → 2 pairs: 2^8 candidates left after the first)
Forge MAC tag by guessing	2^n
HMAC collision attack	2^(n/2) blocks observed online under one key (MD5: ~300,000 years on 1 Gbps)
Dictionary with salt	(#words) × (#salts) hashes
Rainbow table	(#passwords) × 2^(salt bits) entries

C. Constants and full tables

All tables below were generated and checked against the slide values: S(95) = 2A, S(29) = A5, IS(95) = AD, BA × 0B = 72, inv(22) = 5A. Read a byte as row = high nibble, column = low nibble.

AES S-box (SubBytes)

S	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB
1x	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72
2x	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31
3x	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2
4x	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F
5x	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58
6x	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F
7x	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3
8x	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19
9x	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B
Ax	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4

S	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
Bx	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE
Cx	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B
Dx	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D
Ex	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28
Fx	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB

AES inverse S-box (InvSubBytes)

IS	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7
1x	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9
2x	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3
3x	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1
4x	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6
5x	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D
6x	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45
7x	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A
8x	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6
9x	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF
Ax	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE
Bx	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A
Cx	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC
Dx	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C
Ex	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99
Fx	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C

AES internals

- S-box construction: b = inverse of a in GF(2^8) (0 → 0), then S(a) = b ⊕ rotl(b,1) ⊕ rotl(b,2) ⊕ rotl(b,3) ⊕ rotl(b,4) ⊕ 0x63. S(00) = 63.
- MixColumns matrix (rows): `02 03 01 01` / `01 02 03 01` / `01 01 02 03` / `03 01 01 02`.
- InvMixColumns matrix (rows): `0E 0B 0D 09` / `09 0E 0B 0D` / `0D 09 0E 0B` / `0B 0D 09 0E`.
- MixColumns test vector: column DB 13 53 45 → 8E 4D A1 BC.
- ShiftRows: row 0 unchanged, row 1 left 1, row 2 left 2, row 3 left 3. InvShiftRows rotates right.
- Rcon / RC[j], j = 1…14: `01 02 04 08 10 20 40 80 1B 36 6C D8 AB 4D` (AES-128 uses the first 10).
- Key expansion (AES-128): w[i] = w[i−4] ⊕ temp; temp = w[i−1], or for i mod 4 = 0: SubWord(RotWord(w[i−1])) ⊕ Rcon[i/4]. RotWord [B0,B1,B2,B3] → [B1,B2,B3,B0]. Round key r = w[4r..4r+3].
- AES-128 test vector (FIPS-197): key 000102030405060708090a0b0c0d0e0f, plaintext 00112233445566778899aabbccddeeff → ciphertext 69c4e0d86a7b0430d8cdb78070b4c55a.

- GF(2^8) modulus m(x) = x^8 + x^4 + x^3 + x + 1 = 0x11B; xtime(a) = (a << 1) ⊕ (0x1B if a & 0x80).
- GF(2^128) (XTS, GCM, CMAC-AES): x^128 + x^7 + x^2 + x + 1 → reduction constant 0x87. GF(2^64) (CMAC-3DES): x^64 + x^4 + x^3 + x + 1 → 0x1B.
- Irreducible polynomials for n = 3: x^3 + x^2 + 1 and x^3 + x + 1.

GF(2^8) multiply by 02 (xtime)

×02	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	02	04	06	08	0A	0C	0E	10	12	14	16	18	1A	1C
1x	20	22	24	26	28	2A	2C	2E	30	32	34	36	38	3A	3C
2x	40	42	44	46	48	4A	4C	4E	50	52	54	56	58	5A	5C
3x	60	62	64	66	68	6A	6C	6E	70	72	74	76	78	7A	7C
4x	80	82	84	86	88	8A	8C	8E	90	92	94	96	98	9A	9C
5x	A0	A2	A4	A6	A8	AA	AC	AE	B0	B2	B4	B6	B8	BA	BC
6x	C0	C2	C4	C6	C8	CA	CC	CE	D0	D2	D4	D6	D8	DA	DC
7x	E0	E2	E4	E6	E8	EA	EC	EE	F0	F2	F4	F6	F8	FA	FC
8x	1B	19	1F	1D	13	11	17	15	0B	09	0F	0D	03	01	07
9x	3B	39	3F	3D	33	31	37	35	2B	29	2F	2D	23	21	27
Ax	5B	59	5F	5D	53	51	57	55	4B	49	4F	4D	43	41	47
Bx	7B	79	7F	7D	73	71	77	75	6B	69	6F	6D	63	61	67
Cx	9B	99	9F	9D	93	91	97	95	8B	89	8F	8D	83	81	87
Dx	BB	B9	BF	BD	B3	B1	B7	B5	AB	A9	AF	AD	A3	A1	A7
Ex	DB	D9	DF	DD	D3	D1	D7	D5	CB	C9	CF	CD	C3	C1	C7
Fx	FB	F9	FF	FD	F3	F1	F7	F5	EB	E9	EF	ED	E3	E1	E7

GF(2^8) multiply by 03

×03	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	03	06	05	0C	0F	0A	09	18	1B	1E	1D	14	17	12
1x	30	33	36	35	3C	3F	3A	39	28	2B	2E	2D	24	27	22
2x	60	63	66	65	6C	6F	6A	69	78	7B	7E	7D	74	77	72
3x	50	53	56	55	5C	5F	5A	59	48	4B	4E	4D	44	47	42
4x	C0	C3	C6	C5	CC	CF	CA	C9	D8	DB	DE	DD	D4	D7	D2
5x	F0	F3	F6	F5	FC	FF	FA	F9	E8	EB	EE	ED	E4	E7	E2
6x	A0	A3	A6	A5	AC	AF	AA	A9	B8	BB	BE	BD	B4	B7	B2
7x	90	93	96	95	9C	9F	9A	99	88	8B	8E	8D	84	87	82
8x	9B	98	9D	9E	97	94	91	92	83	80	85	86	8F	8C	89
9x	AB	A8	AD	AE	A7	A4	A1	A2	B3	B0	B5	B6	BF	BC	B9
Ax	FB	F8	FD	FE	F7	F4	F1	F2	E3	E0	E5	E6	EF	EC	E9
Bx	CB	C8	CD	CE	C7	C4	C1	C2	D3	D0	D5	D6	DF	DC	D9
Cx	5B	58	5D	5E	57	54	51	52	43	40	45	46	4F	4C	49
Dx	6B	68	6D	6E	67	64	61	62	73	70	75	76	7F	7C	79
Ex	3B	38	3D	3E	37	34	31	32	23	20	25	26	2F	2C	29
Fx	0B	08	0D	0E	07	04	01	02	13	10	15	16	1F	1C	19

GF(2^8) multiply by 09 (InvMixColumns)

×09	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	09	12	1B	24	2D	36	3F	48	41	5A	53	6C	65	7E
1x	90	99	82	8B	B4	BD	A6	AF	D8	D1	CA	C3	FC	F5	EE
2x	3B	32	29	20	1F	16	0D	04	73	7A	61	68	57	5E	45
3x	AB	A2	B9	B0	8F	86	9D	94	E3	EA	F1	F8	C7	CE	D5
4x	76	7F	64	6D	52	5B	40	49	3E	37	2C	25	1A	13	08
5x	E6	EF	F4	FD	C2	CB	D0	D9	AE	A7	BC	B5	8A	83	98
6x	4D	44	5F	56	69	60	7B	72	05	0C	17	1E	21	28	33
7x	DD	D4	CF	C6	F9	F0	EB	E2	95	9C	87	8E	B1	B8	A3
8x	EC	E5	FE	F7	C8	C1	DA	D3	A4	AD	B6	BF	80	89	92
9x	7C	75	6E	67	58	51	4A	43	34	3D	26	2F	10	19	02
Ax	D7	DE	C5	CC	F3	FA	E1	E8	9F	96	8D	84	BB	B2	A9
Bx	47	4E	55	5C	63	6A	71	78	0F	06	1D	14	2B	22	39
Cx	9A	93	88	81	BE	B7	AC	A5	D2	DB	C0	C9	F6	FF	E4
Dx	0A	03	18	11	2E	27	3C	35	42	4B	50	59	66	6F	74
Ex	A1	A8	B3	BA	85	8C	97	9E	E9	E0	FB	F2	CD	C4	DF
Fx	31	38	23	2A	15	1C	07	0E	79	70	6B	62	5D	54	4F

GF(2^8) multiply by 0B (InvMixColumns)

×0B	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	0B	16	1D	2C	27	3A	31	58	53	4E	45	74	7F	62
1x	B0	BB	A6	AD	9C	97	8A	81	E8	E3	FE	F5	C4	CF	D2
2x	7B	70	6D	66	57	5C	41	4A	23	28	35	3E	0F	04	19
3x	CB	C0	DD	D6	E7	EC	F1	FA	93	98	85	8E	BF	B4	A9
4x	F6	FD	E0	EB	DA	D1	CC	C7	AE	A5	B8	B3	82	89	94
5x	46	4D	50	5B	6A	61	7C	77	1E	15	08	03	32	39	24
6x	8D	86	9B	90	A1	AA	B7	BC	D5	DE	C3	C8	F9	F2	EF
7x	3D	36	2B	20	11	1A	07	0C	65	6E	73	78	49	42	5F
8x	F7	FC	E1	EA	DB	D0	CD	C6	AF	A4	B9	B2	83	88	95
9x	47	4C	51	5A	6B	60	7D	76	1F	14	09	02	33	38	25
Ax	8C	87	9A	91	A0	AB	B6	BD	D4	DF	C2	C9	F8	F3	EE
Bx	3C	37	2A	21	10	1B	06	0D	64	6F	72	79	48	43	5E
Cx	01	0A	17	1C	2D	26	3B	30	59	52	4F	44	75	7E	63
Dx	B1	BA	A7	AC	9D	96	8B	80	E9	E2	FF	F4	C5	CE	D3
Ex	7A	71	6C	67	56	5D	40	4B	22	29	34	3F	0E	05	18
Fx	CA	C1	DC	D7	E6	ED	F0	FB	92	99	84	8F	BE	B5	A8

GF(2^8) multiply by 0D (InvMixColumns)

×0D	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	0D	1A	17	34	39	2E	23	68	65	72	7F	5C	51	46
1x	D0	DD	CA	C7	E4	E9	FE	F3	B8	B5	A2	AF	8C	81	96
2x	BB	B6	A1	AC	8F	82	95	98	D3	DE	C9	C4	E7	EA	FD
3x	6B	66	71	7C	5F	52	45	48	03	0E	19	14	37	3A	2D
4x	6D	60	77	7A	59	54	43	4E	05	08	1F	12	31	3C	2B
5x	BD	B0	A7	AA	89	84	93	9E	D5	D8	CF	C2	E1	EC	FB
6x	D6	DB	CC	C1	E2	EF	F8	F5	BE	B3	A4	A9	8A	87	90

×0D	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
7x	06	0B	1C	11	32	3F	28	25	6E	63	74	79	5A	57	40
8x	DA	D7	C0	CD	EE	E3	F4	F9	B2	BF	A8	A5	86	8B	9C
9x	0A	07	10	1D	3E	33	24	29	62	6F	78	75	56	5B	4C
Ax	61	6C	7B	76	55	58	4F	42	09	04	13	1E	3D	30	27
Bx	B1	BC	AB	A6	85	88	9F	92	D9	D4	C3	CE	ED	E0	F7
Cx	B7	BA	AD	A0	83	8E	99	94	DF	D2	C5	C8	EB	E6	F1
Dx	67	6A	7D	70	53	5E	49	44	0F	02	15	18	3B	36	21
Ex	0C	01	16	1B	38	35	22	2F	64	69	7E	73	50	5D	4A
Fx	DC	D1	C6	CB	E8	E5	F2	FF	B4	B9	AE	A3	80	8D	9A

GF(2^8) multiply by 0E (InvMixColumns)

×0E	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	0E	1C	12	38	36	24	2A	70	7E	6C	62	48	46	54
1x	E0	EE	FC	F2	D8	D6	C4	CA	90	9E	8C	82	A8	A6	B4
2x	DB	D5	C7	C9	E3	ED	FF	F1	AB	A5	B7	B9	93	9D	8F
3x	3B	35	27	29	03	0D	1F	11	4B	45	57	59	73	7D	6F
4x	AD	A3	B1	BF	95	9B	89	87	DD	D3	C1	CF	E5	EB	F9
5x	4D	43	51	5F	75	7B	69	67	3D	33	21	2F	05	0B	19
6x	76	78	6A	64	4E	40	52	5C	06	08	1A	14	3E	30	22
7x	96	98	8A	84	AE	A0	B2	BC	E6	E8	FA	F4	DE	D0	C2
8x	41	4F	5D	53	79	77	65	6B	31	3F	2D	23	09	07	15
9x	A1	AF	BD	B3	99	97	85	8B	D1	DF	CD	C3	E9	E7	F5
Ax	9A	94	86	88	A2	AC	BE	B0	EA	E4	F6	F8	D2	DC	CE
Bx	7A	74	66	68	42	4C	5E	50	0A	04	16	18	32	3C	2E
Cx	EC	E2	F0	FE	D4	DA	C8	C6	9C	92	80	8E	A4	AA	B8
Dx	0C	02	10	1E	34	3A	28	26	7C	72	60	6E	44	4A	58
Ex	37	39	2B	25	0F	01	13	1D	47	49	5B	55	7F	71	63
Fx	D7	D9	CB	C5	EF	E1	F3	FD	A7	A9	BB	B5	9F	91	83

GF(2^8) multiplicative inverse (mod 0x11B; inv(00) defined as 00)

inv	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
0x	00	01	8D	F6	CB	52	7B	D1	E8	4F	29	C0	B0	E1	E5
1x	74	B4	AA	4B	99	2B	60	5F	58	3F	FD	CC	FF	40	EE
2x	3A	6E	5A	F1	55	4D	A8	C9	C1	0A	98	15	30	44	A2
3x	2C	45	92	6C	F3	39	66	42	F2	35	20	6F	77	BB	59
4x	1D	FE	37	67	2D	31	F5	69	A7	64	AB	13	54	25	E9
5x	ED	5C	05	CA	4C	24	87	BF	18	3E	22	F0	51	EC	61
6x	16	5E	AF	D3	49	A6	36	43	F4	47	91	DF	33	93	21
7x	79	B7	97	85	10	B5	BA	3C	B6	70	D0	06	A1	FA	81
8x	83	7E	7F	80	96	73	BE	56	9B	9E	95	D9	F7	02	B9
9x	DE	6A	32	6D	D8	8A	84	72	2A	14	9F	88	F9	DC	89
Ax	FB	7C	2E	C3	8F	B8	65	48	26	C8	12	4A	CE	E7	D2
Bx	0C	E0	1F	EF	11	75	78	71	A5	8E	76	3D	BD	BC	86
Cx	0B	28	2F	A3	DA	D4	E4	0F	A9	27	53	04	1B	FC	AC
Dx	7A	07	AE	63	C5	DB	E2	EA	94	8B	C4	D5	9D	F8	90
Ex	B1	0D	D6	EB	C6	0E	CF	AD	08	4E	D7	E3	5D	50	1E

inv	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
Fx	5B	23	38	34	68	46	03	8C	DD	9C	7D	A0	CD	1A	41

Stream cipher constants

- Salsa20/ChaCha20 constant "expand 32-byte k" (256-bit key): bytes `65 78 70 61 | 6e 64 20 33 | 32 2d 62 79 | 74 65 20 6b`; as little-endian words `61707865 3320646e 79622d32 6b206574`.
- "expand 16-byte k" (128-bit key): bytes `65 78 70 61 | 6e 64 20 31 | 36 2d 62 79 | 74 65 20 6b`; words `61707865 3120646e 79622d36 6b206574`.
- Salsa20 counter is little-endian: position 17 = bytes `11 00 00 00 00 00 00 00`.
- RC4: S initialised to 0..255, T[i] = K[i mod keylen]; test vector key "Key" → keystream eb9f7781b734ca72a719.

Hash constants

Hash	Initial value (big-endian words unless noted)
MD5	A = 67452301, B = efcdab89, C = 98badcfe, D = 10325476 (stored little-endian)
SHA-1	67452301 efcdab89 98badcfe 10325476 c3d2e1f0
SHA-256	6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19
SHA-224	c1059ed8 367cd507 3070dd17 f70e5939 ffc00b31 68581511 64f98fa7 befa4fa4
SHA-512	6a09e667f3bcc908 bb67ae8584caa73b 3c6ef372fe94f82b a54ff53a5f1d36f1 510e527fade682d1 9b05688c2b3e6c1f 1f83d9abfb41bd6b 5be0cd19137e2179

- SHA-256/512 IV = first 32/64 bits of the fractional parts of √ of the first 8 primes; round constants K_t = fractional parts of ∛ of the first 64/80 primes. SHA-256 K_0 = 428a2f98, K_1 = 71374491.
- SHA-256 functions: Σ0 = ROTR2⊕ROTR13⊕ROTR22, Σ1 = ROTR6⊕ROTR11⊕ROTR25, σ0 = ROTR7⊕ROTR18⊕SHR3, σ1 = ROTR17⊕ROTR19⊕SHR10.
- SHA-512 functions: Σ0 = ROTR28⊕ROTR34⊕ROTR39, Σ1 = ROTR14⊕ROTR18⊕ROTR41, σ0 = ROTR1⊕ROTR8⊕SHR7, σ1 = ROTR19⊕ROTR61⊕SHR6.
- Ch(e,f,g) = (e∧f)⊕(¬e∧g) ("if e then f else g"); Maj(a,b,c) = (a∧b)⊕(a∧c)⊕(b∧c).
- HMAC: ipad = 0x36 × b/8, opad = 0x5C × b/8; b = 512 bits (64 B) for MD5/SHA-1/SHA-256, 1024 bits (128 B) for SHA-384/512; keys longer than b are hashed first.

Padding formats

Where	Format
MD5 / SHA-1 / SHA-256	0x80, zeros until length ≡ 56 mod 64 bytes, 8-byte bit length (MD5 little-endian, SHA big-endian)
SHA-384 / SHA-512	0x80, zeros until length ≡ 112 mod 128 bytes (896 mod 1024 bits), 16-byte big-endian bit length; 1 to 1024 padding bits, always added
SHA-3 (FIPS 202)	pad10*1 with domain bits: append 0x06 … 0x80 (0x86 if it is a single byte) to fill r bits
Keccak (original)	pad10*1: 0x01 … 0x80
PKCS#7 (block ciphers)	n bytes of value n, 1 ⩽ n ⩽ block size; a full block 10 10 … 10 (16 × 0x10) if already aligned
CMAC / ISO 7816-4	0x80 then zeros (1 then 0 bits); uses K2 instead of K1
GCM J0	96-bit IV ‖ 00000001

D. Lookup tables

Letter numbering (classical ciphers, mod 26)

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

- Inverses mod 26 (exist only for odd numbers ≠ 13): 1→1, 3→9, 5→21, 7→15, 9→3, 11→19, 15→7, 17→23, 19→11, 21→5, 23→17, 25→25.
- Caesar decrypt: p = (c − k) mod 26. ROT13 = shift 13 (self-inverse). Atbash: a↔z (p = 25 − c).
- English frequency order: e t a o i n s h r d l c u m w f g y p b v k j x q z. Common bigrams: th he in er an re on at en nd. Trigrams: the and ing ion tio ent. Doubles: ll ee ss oo tt ff.
- Common short words: a, I (1); of to in it is be as at so we he by or on do if me my up an go no us am (2); the and for are but not you all any can had her was one our out day get has him his how man new now old see two way who boy did its let put say she too use (3).

ASCII (row = high nibble, column = low nibble; 0x20 = space)

hex	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
2x	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.

hex	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^
6x	backtick	a	b	c	d	e	f	g	h	i	j	k	l	m	n
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~

- Control: 00 NUL, 09 TAB, 0A LF (\n), 0D CR (\r), 1B ESC, 7F DEL. Printable range 0x20–0x7E.
- Upper ↔ lower case = XOR 0x20 (bit 5). Letter ⊕ space flips case: in m1 ⊕ m2, a byte in 0x41–0x7A often means one side was a space.
- Digits '0'–'9' = 0x30–0x39 (digit = byte − 0x30).

Encodings

Encoding	Alphabet / look	Length rule	Python
Hex (base16)	0-9 a-f	2 chars per byte	`bytes.fromhex(s)`, `b.hex()`
Base64	A-Z a-z 0-9 + / and = padding	4 chars per 3 bytes	`base64.b64decode(s)`
Base64url	A-Z a-z 0-9 - _	same, padding often dropped	`base64.urlsafe_b64decode(s + '==')`
Base32	A-Z 2-7 and = padding	8 chars per 5 bytes	`base64.b32decode(s)`
Base85 / Ascii85	85 printable chars, often `<~ ~>`	5 chars per 4 bytes	`base64.a85decode(s)`
URL encoding	%XX	—	`urllib.parse.unquote(s)`
Binary	0/1 in groups of 8	8 per byte	`int(s, 2)`
Decimal bytes	0–255 numbers	—	`bytes(map(int, s.split()))`
Big integer	long decimal	—	`n.to_bytes((n.bit_length() + 7) // 8, 'big')`

- Base64 index: A–Z = 0–25, a–z = 26–51, 0–9 = 52–61, + = 62, / = 63. Recognisable starts: "eyJ" = '{"' (JSON/JWT), "iVBORw0KGgo" = PNG, "JVBERi" = %PDF, "UEsDB" = ZIP, "/9j/" = JPEG, "Q1RG" = CTF, "ZmxhZ" = flag.
- File magic bytes: PNG 89 50 4E 47 0D 0A 1A 0A; JPEG FF D8 FF; GIF 47 49 46 38; PDF 25 50 44 46 (%PDF); ZIP 50 4B 03 04 (PK); ELF 7F 45 4C 46; gzip 1F 8B.

Identify a hash

Hex length	Bits	Likely
8	32	CRC32, Adler-32
16	64	MySQL < 4.1, truncated hash
32	128	MD5, NTLM, MD4
40	160	SHA-1, RIPEMD-160
56	224	SHA-224, SHA3-224, SHA-512/224
64	256	SHA-256, SHA3-256, SHA-512/256, BLAKE2s
96	384	SHA-384, SHA3-384
128	512	SHA-512, SHA3-512, BLAKE2b, Whirlpool

Prefix	Format
`$1$`	md5crypt
`$2a$`, `$2b$`, `$2y$`	bcrypt (cost after next $)
`$5$`	sha256crypt
`$6$`	sha512crypt (Linux /etc/shadow)
`$argon2id$`	Argon2id
`$y$`	yescrypt
`pbkdf2_sha256$`	Django PBKDF2
`{SHA}`	base64 SHA-1 (LDAP/htpasswd)

Hashes of common strings (unsalted)

Input	MD5	SHA-1	SHA-256
(empty)	d41d8cd98f00b204e9800998ecf8427e	da39a3ee5e6b4b0d3255bfef95601890afd80709	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
a	0cc175b9c0f1b6a831c399e269772661	86f7e437faa5a7fce15d1ddcb9eaeaea377667b8	ca978112ca1bbdcafac231b39a23dc4da786eff8147c4e72b9807785afee48bb
abc	900150983cd24fb0d6963f7d28e17f72	a9993e364706816aba3e25717850c26c9cd0d89d	ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
password	5f4dcc3b5aa765d61d8327deb882cf99	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
123456	e10adc3949ba59abbe56e057f20f883e	7c4a8d09ca3762af61e59520943dc26494f8941b	8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92
12345678	25d55ad283aa400af464c76d713c07ad	7c222fb2927d828af22f592134e8932480637c0d	ef797c8118f02dfb649607dd5d3f8c7623048c9c063d532cc95c5ed7a898a64f
123456789	25f9e794323b453885f5181f1b624d0b	f7c3bc1d808e04732adf679965ccc34ca7ae3441	15e2b0d3c33891ebb0f1ef609ec419420c20e320ce94c65fbc8c3312448eb225

Input	MD5	SHA-1	SHA-256
admin	21232f297a57a5a743894a0e4a801fc3	d033e22ae348aeb5660fc2140aec35850c4da997	8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918
letmein	0d107d09f5bbe40cade3de5c71e9e9b7	b7a875fc1ea228b9061041b7cec4bd3c52ab3ce3	1c8bfe8f801d79745c4631d09fff36c82aa37fc4cce4fc946683d7b336b63032
qwerty	d8578edf8458ce06fbc5bb76a58c5ca4	b1b3773a05c0ed0176787a4f1574ff0075f7521e	65e84be33532fb784c48129675f9eff3a682b27168c0ea744b2cf58ee02337c5
iloveyou	f25a2fc72690b780b2a14e140ef6a9e0	ee8d8728f435fd550f83852aabab5234ce1da528	e4ad93ca07acb8d908a3aa41e920ea4f4ef4f26e7f86cf8291c5db289780a5ae
hello	5d41402abc4b2a76b9719d9110117c592	aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d	2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
test	098f6bcd4621d373cade4e832627b4f6	a94a8fe5ccb19ba61c4c0873d391e987982fbbd3	9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08
root	63a9f0ea7bb98050796b649e85481845	dc76e9f0c0006e8f919e0c515c66dbba3982f785	4813494d137e1631bba301d5acab6e7bb7aa74ce1185d456565ef51d737677b2
flag	327a6c4304ad5938eaf0efb6cc3e53dc	112f3a99b283a4e1788dedd8e0e5d35375c33747	807d0fbcae7c4b20518d4d85664f6820aafdf936104122c5073e7744c46c4b87

- SHA3-256("") = a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a. SHA-512("") starts cf83e1357eefb8bd… MD5("The quick brown fox jumps over the lazy dog") = 9e107d9d372bb6826bd81d3542a419d6.
- LinkedIn 2012 top 5 (SHA-1, unsalted): 123456 (1,135,936×), linkedin 7728240c80b6bfd450849405e8500d6d207783b6 (207,488×), password (188,380×), 123456789 (149,916×), 12345678 (95,854×). 65% cracked within 2 hours.

Powers of two

2^n	Value	Rough meaning
2^8	256	one byte
2^10	1,024	≈ 10^3
2^16	65,536	16-bit salt/tag
2^20	≈ 1.05 × 10^6	≈ 10^6
2^24	≈ 1.68 × 10^7	3 bytes
2^30	≈ 1.07 × 10^9	≈ 10^9, one second at 1 GHz
2^32	≈ 4.29 × 10^9	4 bytes; birthday on 64-bit hash
2^40	≈ 1.1 × 10^12	
2^56	≈ 7.2 × 10^16	DES key space
2^64	≈ 1.8 × 10^19	birthday on 128-bit hash (MD5)
2^80	≈ 1.2 × 10^24	birthday on SHA-1
2^88.4	≈ 4 × 10^26	26! substitution keys

2^n	Value	Rough meaning
2^128	≈ 3.4 × 10^38	AES-128 key space
2^256	≈ 1.2 × 10^77	AES-256 key space

1 year ≈ 3.156 × 10^7 s ≈ 2^25 s. 1 Gbps = 10^9 bits/s.

E. Course polls and exercises, answered

The slides don't print the poll answers. The answers below are worked out from the slide content; numeric ones were computed.

Polls

Deck	Question	Answer
Intro	Most common password in the 2012 LinkedIn hack?	123456 (1,135,936 times)
Intro	Best way to handle passive security attacks?	Encryption (prevention; passive attacks are hard to detect)
Intro	Grey hole attack: router drops a subset of messages. Which active attack?	Denial of service
Symmetric	Key-space size of a general Caesar cipher (26 letters)?	25 useful keys (26 counting k = 0)
Symmetric	Key-space size of a general substitution cipher?	26! ≈ 4 × 10^26
Symmetric	What type of attack is the frequency attack?	Ciphertext-only (cryptanalysis using plaintext language statistics)
Symmetric	"arvsduklio", Vigenère, 2-letter key: which monoalphabetic ciphers?	Even positions "avdki", odd positions "rsulo"
Symmetric	Can a stream cipher be unconditionally secure?	No: the PRG key is shorter than the message, so at best computationally secure
Symmetric	Shortest key length for the ideal block cipher with n = 4?	⌈log2(16!)⌉ = 45 bits (the naive table encoding uses 16 × 4 = 64 bits)
Symmetric	Which feedback mode is most vulnerable to ciphertext bit errors?	CFB: an error stays in the shift register and corrupts the following segments. OFB/CTR: only the same bit; CBC: 2 blocks

Deck	Question	Answer
Hash	Which applications need second preimage resistance?	Message authentication, digital signatures, intrusion/virus detection (not one-way password files)
Hash	33-word text, each separator 1 or 2 spaces: how many variations?	32 separators → 2^32
Hash	Can a MAC be used as a digital signature?	No: both parties share the key, so no nonrepudiation
Hash	Effort to brute-force a k-bit MAC key (n < k) from one message–tag pair?	2^k, leaving about 2^(k−n) matching keys
Hash	k = 24, n = 16: text–tag pairs needed on average?	2 (first pair leaves 2^8 keys, second leaves ≈ 1)
Hash	Rainbow table: 10^9 passwords of 64 bits, SHA3-256, no salt?	10^9 × (8 + 32 B) = 40 GB
Hash	Same with a 16-bit salt?	× 2^16 = 65,536 → ≈ 2.6 PB (≈ 2.75 PB if each row also stores the 2-byte salt)

Exercises

- **Vigenère**: key "pass" (15 0 18 18), ciphertext "ihwscsowg" → **theanswer**.
- **GF(2^8) multiply**: 0xBA × 0x0B = (x^7+x^5+x^4+x^3+x)(x^3+x+1) = x^10+x^4+x^3+x^2+x → mod m(x), quotient x^2 → x^6+x^5+x^4+x = **0x72**.
- **GF(2^8) inverse** of 0x22 = x^5 + x: r1 = x^3+x+1, q1 = x^3, w1 = x^3; r2 = x^2+1, q2 = x^2+1, w2 = x^5+x^3+1; r3 = 1, q3 = x, w3 = x^6+x^4+x^3+x = **0x5A**.
- **Salsa20 initial state** (C = "expand 16-byte k", K = 4ff69c8e1e610aa5c72875cd8c7c69c8, N = 341f8a2905ca7af8, P = 17). Bytes in matrix order: row 1 `65787061 4ff69c8e 1e610aa5 c72875cd`; row 2 `8c7c69c8 6e642031 341f8a29 05ca7af8`; row 3 `11000000 00000000 362d6279 4ff69c8e`; row 4 `1e610aa5 c72875cd 8c7c69c8 7465206b`. As little-endian word values: `61707865 8e9cf64f a50a611e cd7528c7 / c8697c8c 3120646e 298a1f34 f87aca05 / 00000011 00000000 79622d36 8e9cf64f / a50a611e cd7528c7 c8697c8c 6b206574`.
- **Salsa20 quarter round**: QR(E4971786, 0096E9C1, 932F7534, 27F0D9BB) → y0 = **A1477CD4**, y1 = **436E4947**, y2 = **99EEEF64**, y3 = **80E5A210**. Order: y1, y2, y3, then y0.
- **MixColumns**: input rows `87 F2 4D 97 / 6E 4C 90 EC / 46 E7 4A C3 / A6 8C D8 95`. s′0,0 = 2·87 ⊕ 3·6E ⊕ 46 ⊕ A6 = 15 ⊕ B2 ⊕ 46 ⊕ A6 = **47**. Full output rows: `47 40 A3 4C / 37 D4 70 9F / 94 E4 3A 42 / ED A5 A6 BC`.
- **SHA round word** (toy SHA, 8-bit words, 128-bit block, M = 5C 84 C2 03 06 9A AF A7 5A CA 8D 4B C9 2A 06 30): W16 = W0 + σ0(W1) + W9 + σ1(W14) mod 2^8 = 5C + 62 + CA + F0 = 92 + 98 + 202 + 240 = 632 mod 256 = 120 = **0x78**. Here σ0(84) = ROTR1 ⊕ ROTR2 ⊕ SHR7 = 42 ⊕ 21 ⊕ 01 = 62, and σ1(06) = ROTR3 ⊕ ROTR5 ⊕ SHR6 = C0 ⊕ 30 ⊕ 00 = F0.

- **HMAC-SHA256 collision time** (256-bit hash, 512-bit block, 1 Gbps): need 2^128 observed blocks × 512 bits = 2^137 bits → 1.74 × 10^32 s ≈ **5.5 × 10^24 years**. If each observed message is a 1024-byte packet: 2^128 × 8192 bits → ≈ 8.8 × 10^25 years. MD5 reference from the slides: 2^64 × 2^9 = 2^73 bits ≈ 300,000 years.
- **Length-extension example** (slides): key 14 bytes + message 55 bytes = 69 bytes; length field 0x228 = 552 bits; glue = 0x80 + zeros + 00 00 00 00 00 00 02 28.
- **One-time pad example**: message 0110111 ⊕ key 1011001 = ciphertext 1101110.
- **GF(2^8) addition**: 0xBA ⊕ 0x0B = 0xB1.
- **S-box lookups**: S(95) = 2A, S(29) = A5, IS(A5) = 29, IS(95) = AD.